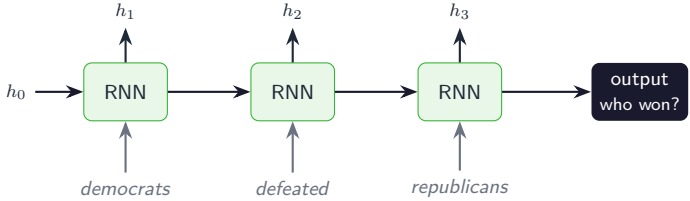
Bags, Vectors & Transformers

Recurrent Neural Networks and LSTMs · Bonus, Part 8

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

Reading Word by Word, with a Memory

A **recurrent neural network** (RNN) reads a text the way you do: one word at a time, updating a **hidden state** h, a vector that summarizes everything read so far.



$h_t = \tanh(Wx_t + Uh_{t-1} + b)$: the **same weights** are used at every step, so an RNN can read texts of any length.

The final memory h summarizes the whole text; an ordinary output layer turns it into a prediction.

An Experiment: When Word Order Is Everything

Generated sentences about a vote; the label is **who won**. Both parties appear in every sentence, so only the *order* gives the answer away.

```
label 1:   on final democrats beat republicans round on the
label 0:   republicans outvoted democrats today round committee
```

Bag of words + logistic regression

52% test accuracy: a coin flip

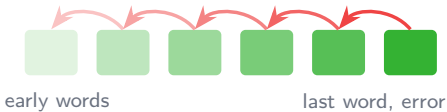
Small LSTM, reading in order

100% test accuracy

Results from the bonus notebook. Word pairs (bigrams) also solve this version, because the clue sits in adjacent words; recurrence matters when clues are far apart.

The Memory Problem

During training, the error signal travels **backward** through every reading step and gets multiplied each time. Over many steps it fades toward zero: the **vanishing gradient**.



Result: a plain RNN mostly remembers the last few words.

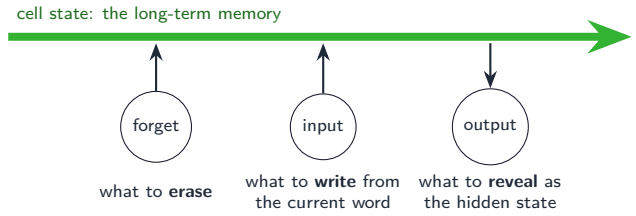
Same task, 8 to 15 filler words after the key part:

plain RNN	50%
LSTM	100%

Test accuracy in the bonus notebook. Training is random: occasionally the plain RNN gets lucky, so the notebook checks several seeds.

How the LSTM Remembers

The **LSTM** (long short-term memory; Hochreiter & Schmidhuber, 1997) adds a protected memory, the **cell state**, that runs through the sequence like a conveyor belt.

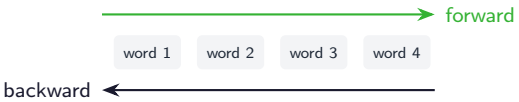


Each **gate** is a small layer with a sigmoid: 0 = closed, 1 = open. Information on the belt only changes when a gate says so, so the error signal survives many steps.

The **GRU** (2014) is a simpler variant with two gates and about 3/4 of the weights. It often works just as well.

Two Practical Tricks

Reading in both directions



A **bidirectional** LSTM runs one LSTM left to right and one right to left and joins their final memories. Every part of the text is close to the end of *some* reading direction, which softens the memory problem.

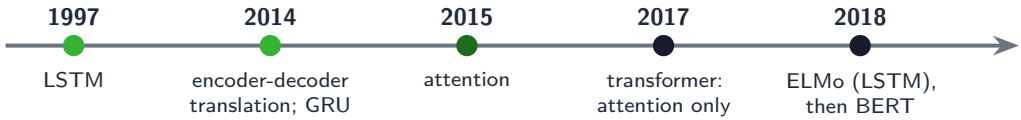
Ignoring the padding

Titles are padded with zeros to equal length. An RNN that reads the padding ends with a memory full of nothing.

```
# Keras:  mark ID 0 as padding
Embedding(V, 50, mask_zero=True)

# PyTorch:  "pack" the true lengths
pack_padded_sequence(emb, lengths, ...)
```

From LSTMs to Attention and Transformers



1. **The bottleneck.** An encoder LSTM squeezes the whole source sentence into one final memory vector, and long sentences don't fit.
2. **Attention** (Bahdanau et al., 2015) lets the decoder look back at *all* hidden states and pick the relevant words. Nothing has to be carried across the sentence anymore.
3. **Transformers** (Vaswani et al., 2017) drop recurrence entirely. All positions are processed in parallel, so models can train on vastly more text: BERT, Day 3.

In the Notebook

- An RNN reading two sentences **by hand**: same words, different memory
- The vote experiment: bag of words vs. an LSTM, and why bigrams also work here
- The memory test: plain RNN vs. LSTM vs. GRU, across several seeds
- A bidirectional LSTM for bill titles in **Keras** and **PyTorch**
- Starting from GloVe, and one vs. two reading directions
- Comparing with the baseline, Bonus 7 and Day 3

`bonus/notebooks/8_rnns_and_lstms_exercises.ipynb` · 6 exercises · CPU is fine